**MINIPROJECT 2**

**NANCY M**
**22/8/25**

AppDbCoontext.cs:

```
using Microsoft.EntityFrameworkCore;
using Day14project.Models;
namespace Day14project.Context
{
    public class AppDbContext:DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        {

        }
        public DbSet<Customer> customerdetails{ get; set; }
        public DbSet<Account> accountdetails { get; set; }
    }
}
```

AccountDetailsController.cs:

```
using Azure.Core;
using Day14project.Context;
using Day14project.Models;
using Day14project.Security;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Net.Http;
using System.Security.Claims;
using System.Text;
using System.Text.Json;

namespace Day14project.Controllers
{
    public record LoginRequest(string Name,string Password);
    [ApiController]
    [Route("[controller]")]

    public class AccountDetailsController : Controller
    {
```

```
        AppDbContext appDbContext;
        Microsoft.Extensions.Options.IOptions<JWTOptions> JwtOptions;
        private readonly HttpClient _httpClient;
        public AccountDetailsController(AppDbContext ctxt,
Microsoft.Extensions.Options.IOptions<JWTOptions> jwtOptions)
        {
            appDbContext = ctxt;
            JwtOptions = jwtOptions;
        }
        [Authorize(Roles ="Admin")]
        [HttpPost("AddCustomer")]
        public IActionResult AddCustomer(Customer customer)
        {
            appDbContext.customerdetails.Add(customer);
            appDbContext.SaveChanges();
            return Ok("customer added");
        }
        [Authorize(Roles = "Admin")]
        [HttpPost("AddAccount")]
        public IActionResult AddAccount(DTOtable dTOtable)
        {
            var customer = appDbContext.customerdetails.Where(x => x.CustomerName ==
dTOtable.CustomerName).FirstOrDefault();
            if (customer == null)
            {
                return NotFound("invalid author");
            }
            Account account = new Account
            {
                AccountNumber = dTOtable.AccountNumber,
                Amount=dTOtable.Amount,
                Date=dTOtable.Date,
                AccountType=dTOtable.AccountType,
                CustomerId=customer.CustomerId,
            };

            appDbContext.accountdetails.Add(account);
            appDbContext.SaveChanges();
            return Ok("added ");
        }

        [HttpGet("GetCustomer")]
        public IActionResult GetCustomer(int id)
        {
```

```
            var customer = appDbContext.customerdetails
                .Where(x => x.CustomerId == id);

            if (customer == null)
            {
                return NotFound("Customer not found");
            }
            return Ok(customer);
        }
        [HttpGet("GetallCustomerorderby")]
        public IActionResult GetallCustomerorderby()
        {
            var allcustomer =
appDbContext.customerdetails.Where(y=>y.CustomerAge>18).OrderByDescending(x =>
x.CustomerAge);
            return Ok(allcustomer);
        }
        const string ADMIN = "admin";
        [HttpPost("Login")]
        public  async Task<IActionResult> Login(LoginRequest Login)
        {
            var HttpClient=new HttpClient();
            var response=await HttpClient.PostAsJsonAsync("http://127.0.0.1:8000/fetch",Login);
            var user =await response.Content.ReadFromJsonAsync<User>();

            if (user == null)
            {
                return Unauthorized("sorry not authorized");
            }

            else
            {
                var claims = new List<Claim>
                {
                     new Claim(ClaimTypes.Name,Login.Name),

                };
               if (user.Role.Trim().ToLower().Contains(ADMIN))
                {
                    claims.Add(new Claim(ClaimTypes.Role, "Admin"));
                }

                var token = Jwtservice.CreateJWTToken(JwtOptions.Value, claims);
                return Ok(token);
```

```
        }
    }
    [HttpGet("GetBalance")]
    public IActionResult GetBalance(string name)
    {
        var customer = appDbContext.accountdetails
            .Where(x => x.customer.CustomerName == name).Sum(y => y.Amount);

        if (customer == null)
        {
            return NotFound("Customer not found");
        }
        return Ok(customer);
    }


}

}
```

Account.cs:

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Day14project.Models
{
    public class Account
    {
        [Key]
        public int AccountNumber { get; set; }
        public int  Amount { get; set; }
        public DateOnly Date { get; set; }
        public int CustomerId { get; set; }
        [ForeignKey("CustomerId")]
        public Customer customer { get; set; }

        public string AccountType { get; set; }
    }
}
```

Customer.cs:

```
using System.ComponentModel.DataAnnotations;
```

```
namespace Day14project.Models
{
    public class Customer
    {
        [Key]
        public int CustomerId { get; set; }
         public string CustomerName { get; set; }
        public int CustomerAge { get; set; }

    }
}
```

DTOtable.cs:

```
namespace Day14project.Models
{
    public class DTOtable
    {
        public int AccountNumber { get; set; }
        public int Amount { get; set; }
        public DateOnly Date { get; set; }

        public string AccountType { get; set; }
        public string CustomerName { get; set; }
    }
}
```

User.cs:

```
namespace Day14project.Models
{
    public class User
    {
        public string Name { get; set; }
        public string Role { get; set; }
    }
}
```

SQLQuery1.sql - P...YODA\nancy.m (55))

```
SELECT TOP (1000) [CustomerId]
      ,[CustomerName]
      ,[CustomerAge]
  FROM [demo_nancy].[dbo].[customerdetails]
```

100 %

Results | Messages

| | CustomerId | CustomerName | CustomerAge |
|---|---|---|---|
| 1 | 1 | jack | 20 |
| 2 | 2 | jill | 10 |
| 3 | 3 | henry | 40 |
| 4 | 4 | dharu | 20 |

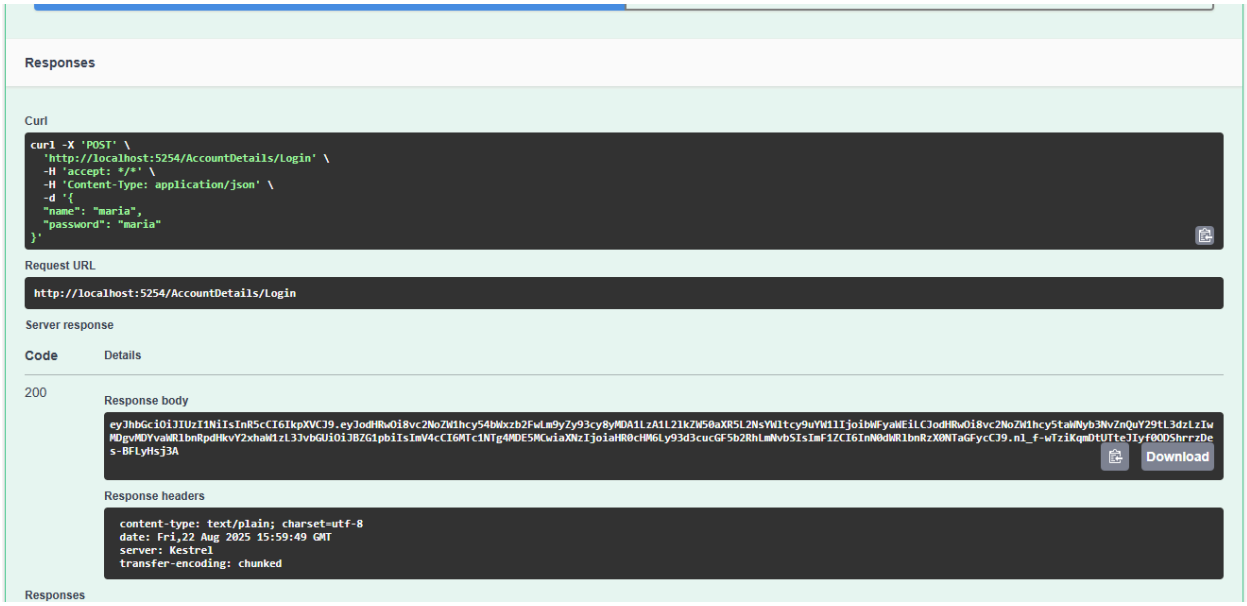
SQLQuery2.sql - P...ODA\nancy.m (105)) | SQLQuery1.sql - P...YODA\nancy.m (55))

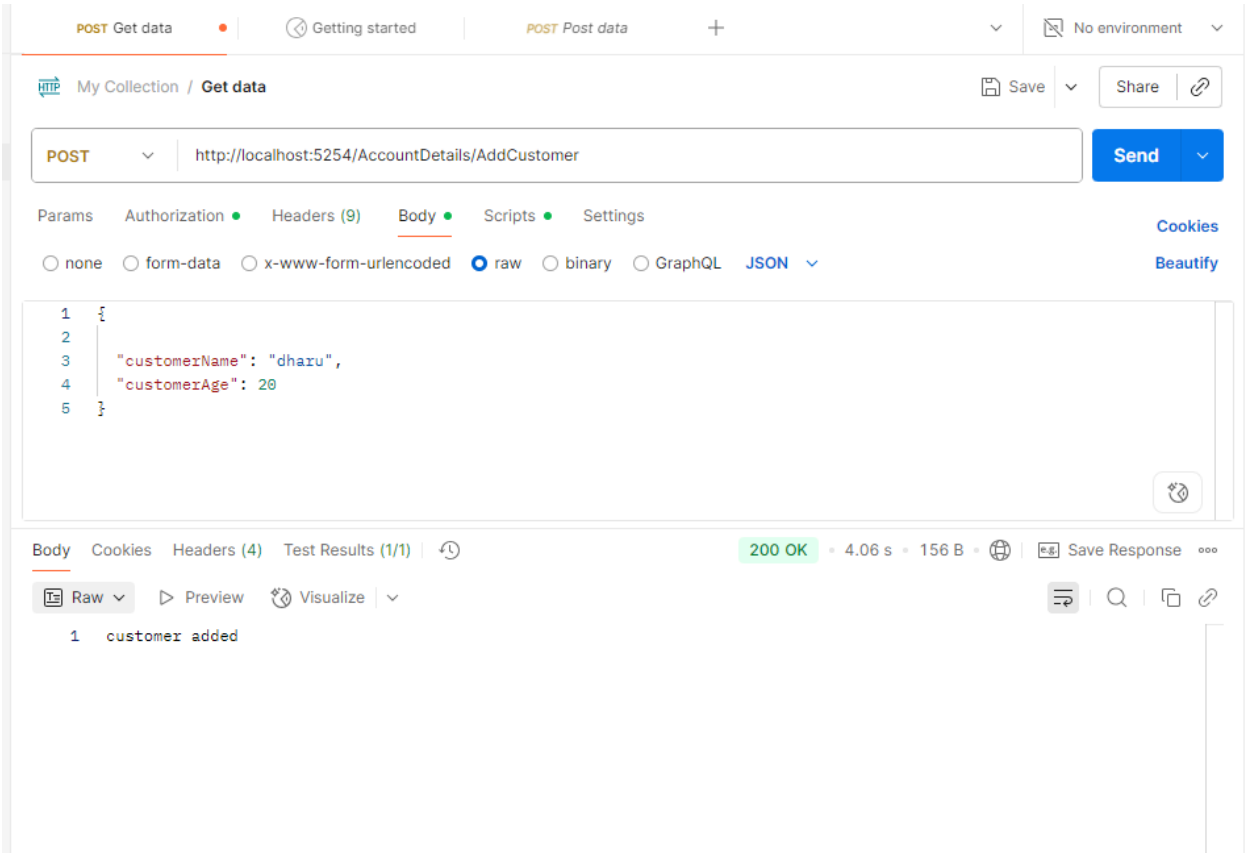
```
SELECT TOP (1000) [AccountNumber]
      ,[Amount]
      ,[Date]
      ,[CustomerId]
      ,[AccountType]
  FROM [demo_nancy].[dbo].[accountdetails]
```

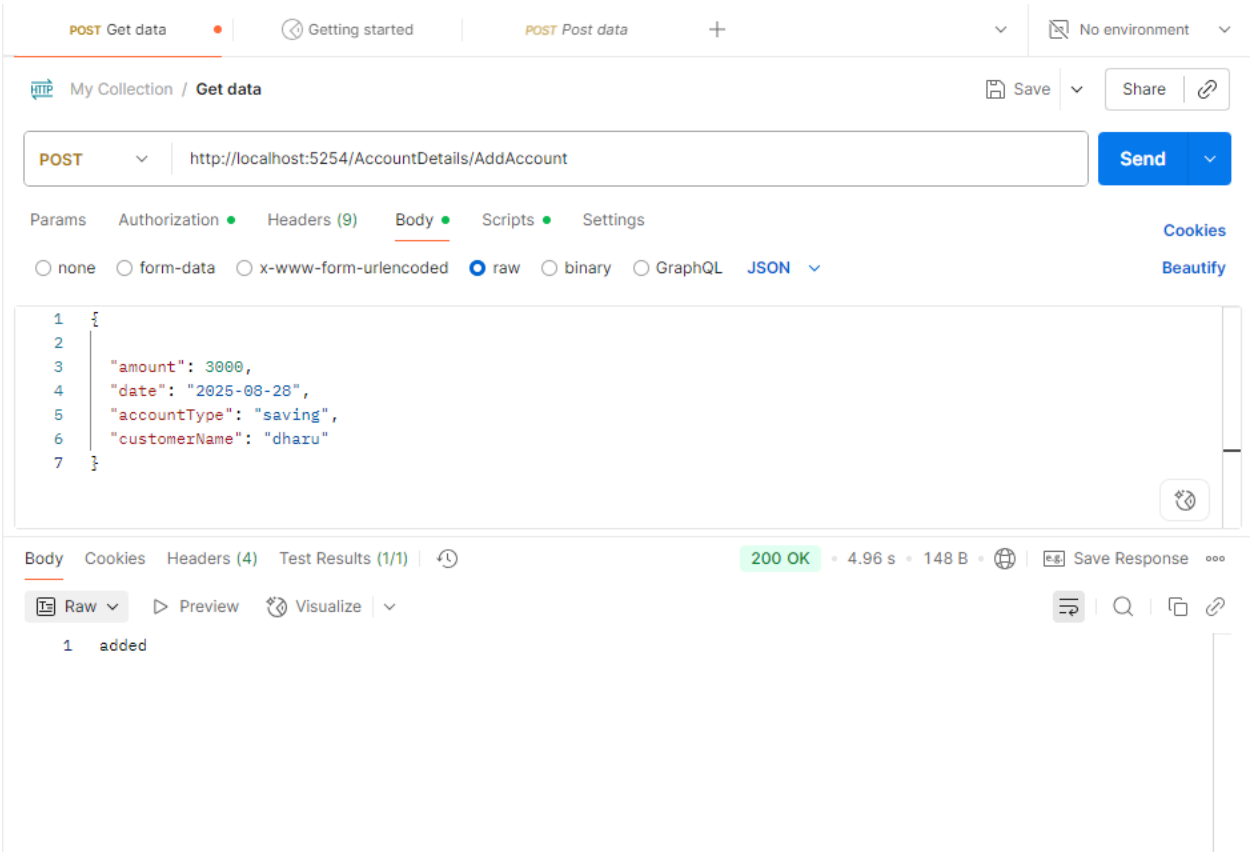
100 %

Results | Messages

| | AccountNumber | Amount | Date | CustomerId | AccountType |
|---|---|---|---|---|---|
| 1 | 1 | 2000 | 2025-08-20 | 1 | 0 |
| 2 | 2 | 3000 | 2025-08-18 | 3 | savings |


Responses
Curl
curl -X 'POST' \
'http://localhost:5254/AccountDetails/Login' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"name": "maria",
"password": "maria"
}'
Request URL
http://localhost:5254/AccountDetails/Login
Server response
Code
Details
200
Response body
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1lIjoibWFyaWEiLCJodHRwOi8vc2NoZW1hcy5taWNyb3NvZnQuY29tL3dzLzIwMDgvMDYvaWRlbnRpdHkvY2xhaW1zL3JvbGUiOiJBZG1pbiIsImV4cCI6MTc1NTg4MDE5MCwiaXNzIjoiaHR0cHM6Ly93d3cucGF5b2RhLmNvbSIsImF1ZCI6InN0dWRlbnRzX0NTaGFycCJ9.nl_f-wTziKqmDtUTteJIyf0ODShrrzDes-BFLyHsj3A
Download
Response headers
content-type: text/plain; charset=utf-8
date: Fri,22 Aug 2025 15:59:49 GMT
server: Kestrel
transfer-encoding: chunked
Responses


POST Get data
Getting started
POST Post data
No environment
My Collection / Get data
Save
Share
POST
http://localhost:5254/AccountDetails/AddCustomer
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"customerName": "dharu",
"customerAge": 20
}
Body
Cookies
Headers (4)
Test Results (1/1)
200 OK
4.06 s
156 B
Save Response
Raw
Preview
Visualize
customer added




Fast.api:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import sessionmaker, Session, declarative_base
from fastapi import FastAPI, Depends, HTTPException, Request
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel
import uvicorn

DATABASE_URL = "sqlite:///./userdetails.db"
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

class User(Base):
    __tablename__ = "userdetails"
    Id = Column(Integer, primary_key=True, index=True)
    Name = Column(String(50), nullable=False)
```

```
    Password = Column(String(50), nullable=False)
    Role = Column(String(50), nullable=False)

Base.metadata.create_all(bind=engine)

app = FastAPI()
templates = Jinja2Templates(directory="templates")

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

class UserSchema(BaseModel):
    Name: str
    Password: str
    Role: str
    class Config:
        from_attributes = True


@app.post("/add_user")
def add_user(user: UserSchema, db: Session = Depends(get_db)):
    new_user = User(Name=user.Name, Password=user.Password, Role=user.Role)
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return new_user

class userrequest(BaseModel):
    name: str
    password: str

@app.post("/fetch")
def get_user(request: userrequest, db: Session = Depends(get_db)):
    usr = db.query(User).filter(User.Name ==
request.name,User.Password==request.password).first()
    return {
        "name": usr.Name,
        "role": usr.Role
    }
```

```
if __name__ == "__main__":
    uvicorn.run("HelloBank:app", host="127.0.0.1", port=8000, reload=True)
```